

Implementation of Fixed-Point FIR Low Pass Filter

Rajesh Reddy

EE24BTECH11043

Department of Electrical Engineering
IIT Hyderabad

1 Introduction

In this experiment, a 101-tap Linear Phase FIR Low Pass Filter was implemented using fixed-point arithmetic and verified against a bit-true MATLAB model.

Three hardware architectures were implemented:

1. Direct Form FIR
2. Symmetry-Based Genvar Adder Tree
3. Optimised Symmetric Sequential FIR

All implementations were verified using Icarus Verilog and compared using a Python post-processing script.

2 FIR Filter Theory

The FIR filter equation is:

$$y[n] = \sum_{k=0}^{100} h[k]x[n-k]$$

Since the filter is linear phase:

$$h[k] = h[100 - k]$$

This symmetry was exploited in two optimized architectures.

3 Fixed-Point Representation

3.1 Input Format (Q3.14)

Inputs and coefficients use:

Q3.14 (17-bit signed)

- 1 sign bit
- 2 integer bits
- 14 fractional bits
- Total = 17 bits

3.2 Multiplication

$$Q3.14 \times Q3.14 \rightarrow Q6.28$$

3.3 Accumulation

Instead of shifting back to Q3.14, the product was preserved and accumulated in:

Q13.28 format

Accumulator width:

41-bit signed

This prevents truncation error and preserves precision.

4 Filter Design in MATLAB

Specifications:

- Sampling Frequency: 10 kHz
- Cutoff Frequency: 1 kHz
- Order: 100
- Taps: 101

```
1 fs = 10000;  
2 fc = 1000;  
3 N = 100;  
4  
5 h = fir1(N, fc/(fs/2), 'low');  
6 h = h / sum(h);
```

Coefficients were quantized to Q3.14 and saved to:

fir_taps_Q3_14.mem

5 Verilog Architectures

5.1 1. Direct Form (lpf_direct.v)

- 101 parallel multipliers

- Full combinational accumulation
- 41-bit accumulator
- Highest hardware cost

5.2 2. Genvar Symmetric Adder Tree (lpf_genvar.v)

Symmetry used:

$$h[k]x[n-k] + h[100-k]x[n-100+k] = h[k](x[n-k] + x[n-100+k])$$

- 50 multipliers
- Balanced adder tree
- Center tap handled separately
- Reduced DSP usage

5.3 3. Optimised Symmetric FIR (lpf_optimised.v)

- 50 multipliers
- Sequential accumulation
- Simplified structure
- 41-bit accumulator

6 Verification Methodology

Three testbenches were implemented:

- tb_lpf_direct.v
- tb_lpf_genvarSS.v
- tb_lpf_optimised.v

Verification steps:

1. Load FIR taps
2. Generate three sine inputs (900 Hz, 1100 Hz, 2000 Hz)

3. Run RTL simulation
4. Save 41-bit Q13.28 output
5. Compare with MATLAB bit-true output

7 Simulation Setup

The designs were simulated using ModelSim/QuestaSim.

7.1 Testbench Design

- Clock period: 10 ns
- Reset applied for first few cycles
- Input samples loaded from memory file
- Coefficients loaded using \$readmemb
- Simulation length: 1000 samples

7.2 Verification Method

- MATLAB used as floating-point reference
- Fixed-point output compared with MATLAB output
- Error calculated between RTL and MATLAB results

8 Simulation Results

8.1 FIR LPF

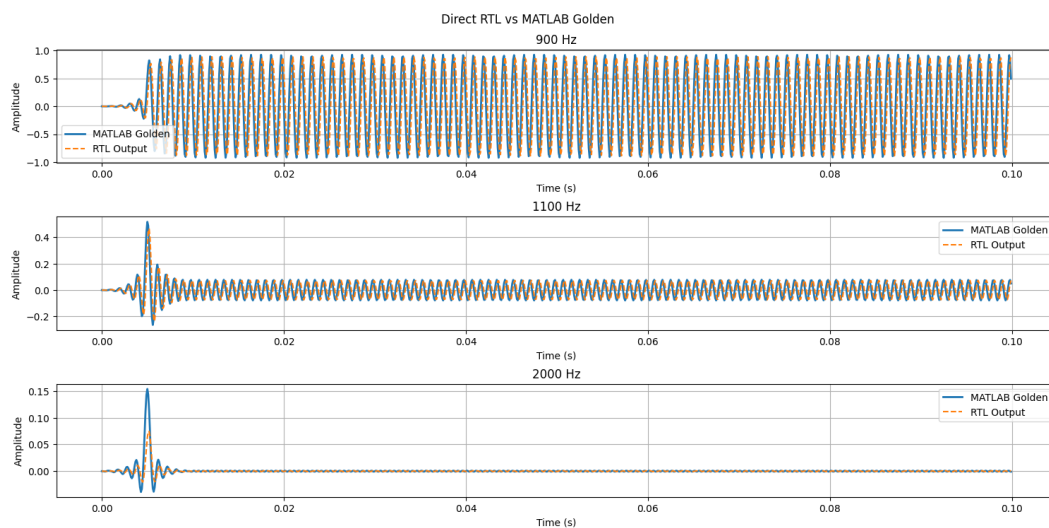


Figure 1: Simulation waveform of FIR LPF

8.2 Genevar LPF

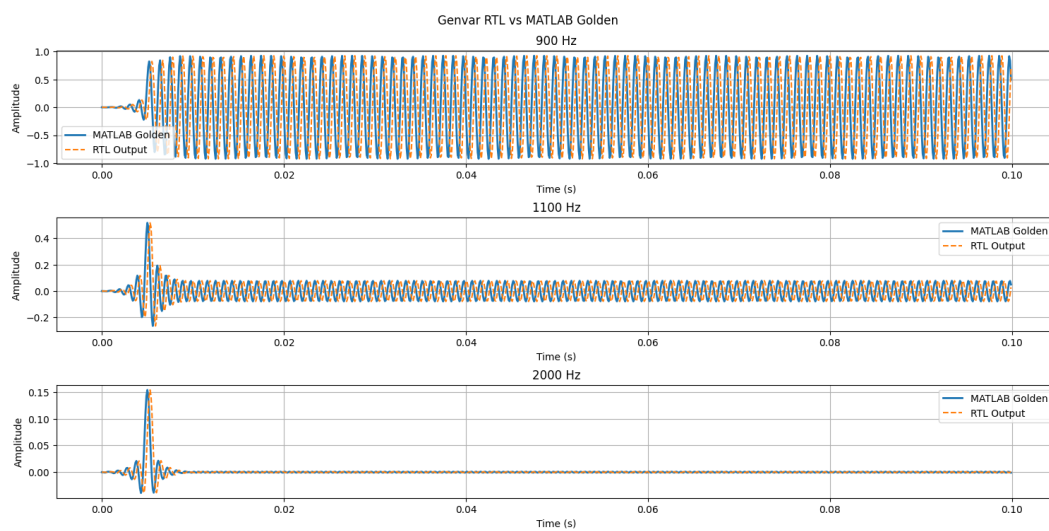


Figure 2: Simulation waveform of Genevar LPF

8.3 Optimized Structural FIR LPF

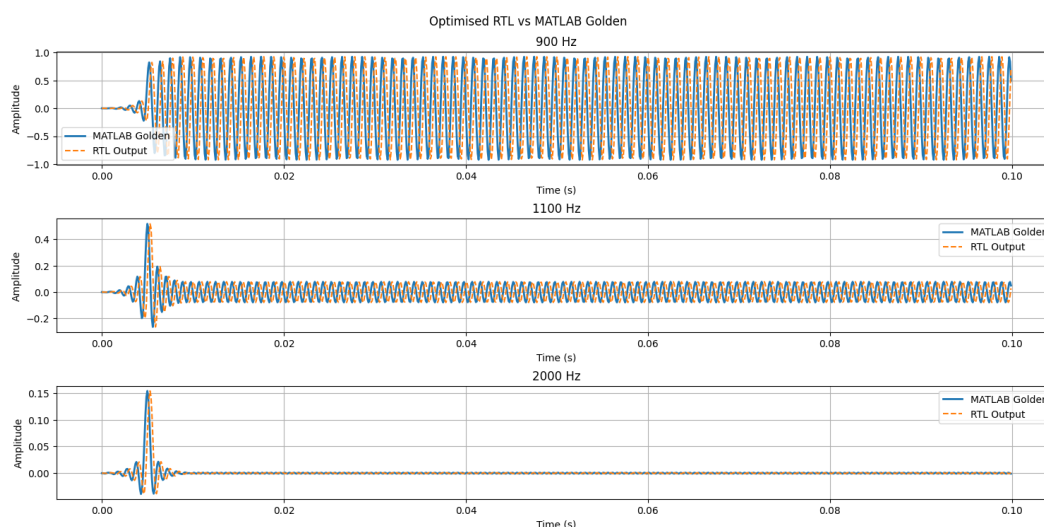


Figure 3: Simulation waveform of Optimized Structural FIR LPF

9 Bit-True MATLAB Model

A bit-true integer model was implemented with:

- Q3.14 inputs
- Q6.28 multiplication
- Q13.28 accumulation
- 41-bit saturation logic

Output file:

lpf_output_Q13_28_41bit.mem

10 Python Post-Processing

Python script:

- Converts 41-bit binary to float
- Aligns MATLAB and RTL outputs
- Plots waveform comparison
- Validates bit-true accuracy

All three designs matched the MATLAB golden output.

11 Architecture Comparison

Architecture	Multipliers	Adders	Hardware Cost
Direct Form	101	100	High
Genvar Symmetric	50	Tree-based	Medium
Optimised Symmetric	50	Sequential	Low

Table 1: Comparison of FIR Architectures

12 Results and Observations

- All three designs match MATLAB bit-true output.
- Symmetry reduces multipliers from 101 to 50.
- Q13.28 accumulation prevents precision loss.
- No overflow observed.
- Adder tree improves timing balance.

13 Conclusion

A 101-tap FIR Low Pass Filter was successfully implemented using three hardware architectures.

All designs:

- Use Q3.14 fixed-point input
- Accumulate in Q13.28 (41-bit)
- Match MATLAB bit-true output
- Demonstrate correct filtering behavior

Symmetry exploitation significantly reduces hardware cost while preserving accuracy.

This experiment demonstrates practical fixed-point DSP implementation and hardware optimization for FPGA systems.